instance. Images that aren't available on the remote instance will be downloaded as needed, just like they were locally.

Introducing Docker Swarm

Remember, we're treating this virtual instance as if it were a production instance for our application. Our objective is to start our application on this instance so that it can be used to service real requests.

There are number of ways you could do this. For example, you *could* use low-level Docker commands to start the various containers for our application. However, having created a nice abstraction for our application with our `docker-compose.yml` file, that doesn't sound appealing. If you're thinking that we could use Compose directly to manage our app on this new instance, you're close to where we're headed.

The truth, though, is that Compose, as a tool, is really designed to help us during *development*. Once we get to a production environment, we have a whole different set of concerns that didn't really affect us in development, such as:

- How do we make our app resilient to failures?
- How can we scale our application to handle varying load?
- How do we deploy new versions of our application with no (or minimal) downtime for users?

Instead, as we saw earlier on page 132, Docker provides a container *orchestrator*, Swarm, for managing apps in production.

Using Swarm (the tool), you create clusters—*swarms*— of one or more connected instances; these work as a single, resilient unit on which to run services in containers. Swarm is declarative: we tell it the state we want our application to be in, and it pulls the various knobs and levers to make this happen. As you'd hope, it leverages the `docker-compose.yml` format that we've used to specify our application.

Our First (Single Node) Swarm

It's time to dive deeper into Swarm and start getting our hands dirty.

Previously, we created a plain-vanilla Docker instance: `local-vm-1`. To turn our instance into a swarm, we have to initialize it explicitly. With our CLI targeting our new instance, we can do this by running:

```
$ docker swarm init --advertise-addr <IP address of instance>
```